

VYDRA CORE

QA Requirements & Fix Status Report

Prepared: August 7, 2026

This document responds to the high-level test findings shared by the QA team. Each reported item is listed against the corresponding fix or verification performed, including the exact files and mechanisms involved, so the status of every point can be independently confirmed.

#	Requirement (as reported)	Status	Work Done / Where Implemented
1	PDF upload is not working as expected. Expectation: an uploaded PDF should generate a quiz (questionnaire with multiple-choice answers) based on the document.	FIXED	Upload & text extraction: backend/app/routers/documents.py (POST /api/documents/upload). Question generation: backend/app/agents/question_generator.py -- tries 3 Groq models in sequence, fails over to Gemini, then to a rule-based fact-extraction generator as a last resort, so a quiz is produced even if AI providers are unavailable. Root cause of the failure seen in testing: Groq's free quota was exhausted <i>and</i> the Gemini key was not yet set on the live server, so both AI paths failed. GEMINI_API_KEY has now been added to the production environment, resolving this.
2	Students should be able to take an assessment for the generated quiz, and it should provide GAP analysis for the student.	DONE	Quiz submission: backend/app/routers/quiz.py (POST /api/quiz/submit-answer). Grading and automatic gap tracking: backend/app/services/quiz_grading.py and backend/app/services/learning_analytics.py (record_gap_snapshot). Results surface in a dedicated Gap Analysis page (frontend/pages/gap-analysis.jsx), linked from the main navigation, showing current weak topics and full history -- backed by backend/app/routers/learning.py (/gaps/topics, /gaps/trend).
3	Scores should be updated correctly for the student assessment.	DONE	backend/app/services/quiz_grading.py (grade_mcq_session) computes total questions and correct answers strictly from what was actually generated for that session -- never trusted from client input -- preventing score manipulation or client/server drift. This single function is shared by both the personal quiz flow and the classroom quiz flow so the two cannot diverge.
4	Dashboard and other sections should be updated correctly.	DONE	frontend/pages/progress.jsx pulls live results from /api/quiz/progress plus /gaps/topics and /gaps/trend on every load. Educator-facing analytics extended the same way in backend/app/routers/educator.py (student-analytics endpoint), so per-student dashboards for teachers reflect the same live gap/score data.
5	Chatbot is providing vague answers even for a simple definition of photosynthesis -- this should be corrected to provide accurate results.	FIXED	Root cause found in backend/app/routers/qa.py (_generate_with_ai): the system prompt previously told the model to answer any "what is X"-style question in "a sentence or two, no padding" -- which collapsed real concept questions into unhelpfully short answers. Prompt rewritten so only truly trivial factual lookups (a date, a name) stay brief; anything asking what a concept or topic is now always gets a complete, accurate explanation.

#	Requirement (as reported)	Status	Work Done / Where Implemented
6	Chatbot should explain chapters (e.g. Periodic Table, Nervous System) in simple, plain English with examples an average student can understand.	FIXED	Same prompt rewrite as item 5, in backend/app/routers/qa.py . The system prompt now explicitly requires: plain, everyday English pitched at a school student, a brief explanation of how/why the concept works, and at least one concrete, relatable example -- for every concept- or chapter-level question, not just when asked.
7	Implement fallback logic so that when Groq's free quota is exhausted, requests automatically route to Google Gemini without breaking the user experience. Groq remains primary; a backend "provider router" detects quota/rate-limit issues and switches to Gemini, which can rotate across multiple project API keys. Frontend stays unchanged; one stable backend endpoint.	DONE	Implemented exactly as specified in backend/app/services/ai_generation.py (ai_chat_completion / ai_json_completion): every request tries Groq first; on any timeout, rate limit, or error it automatically fails over to Gemini, rotating across up to two configured Gemini API keys. The frontend and every other backend module call one stable function and never see which provider actually served the request. Used by: chat answers, quick-check generation, document "Material Intelligence" (summaries/glossary/flashcards), and meeting-summary generation. Quiz generation (question_generator.py) implements the same Groq-primary, Gemini-fallback order independently, since it needed its own multi-model Groq retry logic first. AI image generation is Gemini-only (Groq has no image API), so it goes straight to Gemini with the same 2-key rotation.

Note on Item 7 / provider order: the codebase had briefly been switched to a Gemini-primary configuration during earlier iteration. It has been reverted to match this specification exactly -- Groq primary, automatic Gemini fallback with key rotation -- and committed.

Deployment note: all fixes above are committed to the repository. They still need to be (1) pushed to GitHub, and (2) redeployed on the hosting backend (Render) and frontend (Vercel) for the fixes to take effect on the live application.